

# Team Unagi at ICFPC 2026

A detailed analysis of their per-problem approaches, contest operation, and machinery — compared throughout with team wheezards.

wheezards · 2026-08-05 · sources: [github.com/icfpc-unagi/icfpc2026](https://github.com/icfpc-unagi/icfpc2026) · [github.com/tarstars/icfpc2026-claude](https://github.com/tarstars/icfpc2026-claude)

## Team Unagi at ICFPC 2026: A Detailed Analysis

---

An engineering study of [icfpc-unagi/icfpc2026](https://github.com/icfpc-unagi/icfpc2026), compared throughout with team wheezards' own contest · prepared by the wheezards tooling agent, 2026-08-05

---

### Abstract

Team Unagi — six-time-class ICFPC winners — published their complete ICFP Programming Contest 2026 repository. We cloned it (779 commits, 384 MB, 6 contributors) and read it end to end: every per-problem postmortem trail (~230 documents), the toolchain crates (~45k lines of Rust), the autonomous-agent systems, and the operational documentation. This report reconstructs, in detail: **(1)** their approach to every one of the sixteen graded problems, **(2)** their approach to the contest as an operation, and **(3)** the machinery behind each stage of their pipeline — simulation, generation, composition, optimization, verification, submission, and knowledge management.

Every comparison with our own team (wheezards) is anchored to one moment: our archived standings snapshot of **2026-07-27 08:38 UTC** (~3.5 hours before the end, pre-freeze), the only timestamp at which both teams' numbers are directly comparable. At that moment Unagi was **rank 11 of 268 with 29.31 points**; we were **rank 39 with 24.68**.

The one-sentence conclusion: Unagi ran a three-layer factory — *algorithms chosen and re-chosen by strong humans, machines emitted by per-problem generators over a verified gadget discipline, and a generic annealing optimizer polishing valid artifacts* — on cloud infrastructure that existed before the contest began, with LLM agents embedded as scheduled, leaderboard-driven workers inside that factory. The 4.6-point gap between the teams is concentrated almost entirely in six problems where their algorithm was a different complexity class from ours; where the fight was layout and operations, we were within small factors and once ahead.

## Method and provenance

- Repository cloned 2026-08-05 from `github.com/icfpc-unagi/icfpc2026`.
- All quoted scores, geometries, tick counts, submission IDs and quotes come from files in that repository (postmortems under `solutions/<slug>/`, docs under `docs/`, code under `crates/`, `iwiwi/`, `chokudai/`, `wata/`, `tos/`) or from our own archived contest-site snapshot (`archive/site-snapshot/` in `tarstars/icfpc2026-claude`, captured 2026-07-27 08:38–09:00 UTC).
- Their repository demonstrably lags their final live submissions (their committed best for several problems is worse than the score the official standings showed at 08:38), so per-problem “repo best” and “snapshot official” are both reported where they differ.
- The working notes behind this report are committed as artifacts in `reports/unagi-analysis/notes/` of our repository.

## The contest in one page

ICFPC 2026 (“the basics of real-world systems programming”) ran 2026-07-24 12:00 UTC → 2026-07-27 12:00 UTC. Teams programmed the **littelman machine**: a 2-D ASCII grid of *rooms* connected by *pipes*, in which little men walk over instruction cells. Key mechanics that shape everything below:

- A man walks in a heading; blank cells preserve it; arrows change it; instruction cells execute (`M/W` register moves, arithmetic, `s/r` pipe I/O, branches `X/d/a/x`, literals in backticks, `H` halt).
- `r/s` bind to the **nearest** pipe endpoint (Manhattan, tie by coordinates) — *where* an instruction stands determines *which* pipe it talks to. Geometry is an operand.
- Pipes are FIFOs; **pipe length = latency AND capacity**. Backtick literals pair per row and per column — vertical alignment of unrelated literals can create illegal pairs.
- Displays (LM-75) take address/data/swap streams; some problems are scored on emitted frames.
- **Score (most problems) =  $\max(\text{width}, \text{height})^2 \times \text{average ticks}$**  — lower is better. Footprint is squared, so the long side dominates; ticks stop counting at the last correct output.
- **Points per problem = cases-passed fraction + rank fraction (2.0 max)**; only a team’s best submission counts, so submitting is free and floors are valuable.

## Headline comparison

Both teams at 2026-07-27 08:38 UTC (rank / score; “gap” = our score ÷ theirs; lower score is better):

| problem         | Unagi      | wheelzards        | gap          | nature of the gap            |
|-----------------|------------|-------------------|--------------|------------------------------|
| triangle        | 59 / 832   | 118 / 832         | 1.0×         | shared optimum               |
| history-lesson  | 28 / 6,889 | <b>21 / 6,561</b> | <b>0.95×</b> | <b>we were ahead</b>         |
| memory          | 8 / 8.85M  | 20 / 14.0M        | 1.6×         | layout/protocol polish       |
| tcp             | 30 / 821K  | 32 / 1.36M        | 1.7×         | layout/protocol polish       |
| sort-numbers    | 14 / 320K  | 49 / 802K         | 2.5×         | layout + micro-protocol      |
| reverse-a-list  | 17 / 20.0K | 47 / 84.4K        | 4.2×         | architecture (delay-line)    |
| llm             | 11 / 4.81B | 24 / 21.2B        | 4.4×         | interpreter constants        |
| brackets        | 13 / 83.8K | 41 / 377K         | 4.5×         | footprint class (16 vs 22)   |
| snake           | 21 / 107M  | 36 / 809M         | 7.6×         | generated vs patched         |
| gradebook       | 36 / 405M  | 66 / 47.1B        | 116×         | <b>algorithm</b>             |
| pathfinder      | 33 / 114B  | 54 / 16.1T        | 141×         | <b>algorithm</b>             |
| matmul          | 11 / 16.7M | 58 / 5.93B        | 356×         | <b>algorithm</b>             |
| plotter         | 6 / 3.72M  | 63 / 1.01B        | 271×         | <b>algorithm + codegen</b>   |
| llm             | 27 / 16.1T | 46 / 8.78Q        | 545×         | <b>interpreter constants</b> |
| sudoku-validity | 23 / 2.50M | 74 / 9.29B        | 3,709×       | <b>algorithm</b>             |
| subset-sum      | 11 / 995M  | 61 / 37.4T        | 37,601×      | <b>algorithm class</b>       |

Our final submissions after 08:38 improved several rows (pathfinder 16.1T → 4.37T, gradebook → 34.0B, sudoku → 6.74B, llm → 17.6B), and their committed artifacts show them still descending too. The shape of the table does not change: **six problems ≥100×**, **all algorithmic**; **nine problems ≤8×**, **all layout-and-operations**; **one problem where we led**.

Reading the table strategically: under this contest’s scoring, each problem’s rank fraction is worth up to 1.0 point. The six ≥100× rows are approximately the entire final gap between rank 39 and rank 11. Nothing about our operational execution — judging, submitting, coordinating, squeezing — could have closed them; only different algorithms could. That is the central lesson of this study, and it is also, almost verbatim, the doctrine written at the top of Unagi’s own technique catalog: “*most problems are footprint-tick scored; the two axes often move in opposite directions; always judge by the product*” — their catalog’s preamble, our `claude_44`, independently converged.

# Part I — Their Approach to the Contest

## 1. The team and the division of labor

Six members, all veterans: Takuya Akiba (iwiwi), Kentaro Imajo (imos), Hiroaki Iwami (sulume), Yoichi Iwata (wata), Toshiki Kataoka, Naohiro Takahashi (tos). The commit record (779 commits) shows a clean split:

| member    | commits | primary territory              | inferred role                                            |
|-----------|---------|--------------------------------|----------------------------------------------------------|
| Akiba     | 323     | iwiwi/ (8,613 files)           | autonomous agent systems v1 → v4; post-contest editorial |
| sulume    | 202     | solutions/ (659 files touched) | solution iteration and postmortems                       |
| Imajo     | 132     | crates/ (692), docs/ (634)     | infrastructure, optimizer, documentation system          |
| Kataoka   | 62      | crates/ , tos/ , problems/     | tooling                                                  |
| Iwata     | 49      | solutions/ (453 files)         | hand-crafted flagship solvers ( wata-orz artifacts)      |
| Takahashi | 7       | tos/ diaries                   | artisan solving with plain-text diaries                  |

Daily commit cadence: 88 (day 1) → 269 → 280 → 135 (final half-day) — sustained parallel work, not a final-day crunch.

Three personal workflows coexist under shared conventions: Akiba’s agent loops, wata’s prompt-driven sessions with a human handling git, tos’s plain-text diaries ( `tos/gradebook-diary.txt` , `memory-diary.txt` , `sort-numbers-diary.txt` ) plus snapshot scripts. The shared convention that binds them is the **score-named artifact**: `solutions/<slug>/<score>.man` + `<score>.md` + optionally `<score>.py` (generator). “The current best is the numerically smallest score” is literally how wata’s agent prompt defines the interface.

## 2. Preparation: the contest before the contest

`docs/preparation.md` records the doctrine, distilled from their 2023–2025 repos (kept as reference clones): “*make every task-independent thing work in advance, including one full rehearsal on a past problem.*” Ported and verified green before the start:

- A Rust workspace split by dependency into `unagi-util` / `gcp` / `sql` / `lock` / `executor` / `www` plus contest-specific crates to be added later.
- GCP: buckets ( `gs://icfpc2026` , `gs://icfpc2026-data` ), service account, Cloud SQL + phpMyAdmin, Artifact Registry, Cloud Run deploy targets via `make docker/<service>@deploy` , billing alerts — and a **GCE quota request filed early** (they used 224-vCPU machines in 2023; quota raises have lead time).
- Secrets bootstrapped from a single `UNAGI_PASSWORD` : AES-256-CBC encrypted configs committed in-repo, webhook URLs and API keys fetched at runtime from GCS, never committed and never logged. Repo-local `gcloud/gsutil` wrappers install a project-scoped Cloud SDK under `.cache/` on first use — no system-wide installs.
- SAT solvers (cadical, rustsat-\*) pre-declared as opt-in workspace dependencies in case the task wanted them.
- `AGENTS.md` — standing rules for LLM development agents: minimal diffs, tests + `make lint` after Rust changes, never print secrets, GCP mutations require explicit human approval, destructive operations forbidden, update docs when behavior changes.

### 3. Running services during the contest

`docs/services.md` and `docs/api/submissions.md` describe a small production system:

- **server** (Cloud Run, custom domain `icfpc.sx9.jp`, Basic auth): the human dashboard — tasks, locks, communications.
- **api** (Cloud Run): the **submission relay**. `./bin/submit` authenticates with `UNAGI_PASSWORD`; the service stores each program as versioned JSON in GCS, keeps a persistent queue + state + event history in Cloud SQL, and is the *only* place that reads the official contest API key (from GCS, cached 5 minutes in-process). No member laptop ever holds the key. A Cloud Scheduler cron drives `process-one` queue steps.
- **system**: periodic jobs; **Discord webhooks** for notifications (URL fetched at runtime, never embedded).
- **unagi-executor**: the DB-backed task executor that fans work out across a fleet of GCE machines — carried over from previous years.
- A **scoreboard** binary reads their own submission history and the official standings through the `www` API and re-runs public cases through the local simulator — continuous like-for-like scoring.

Compare us: one coordinator agent (Claude) held the API key locally and was the single submitter; judging was a local script (`subdb.py`, `preflight.py`); coordination was git-mediated messages. Functionally similar shape — gatekeeper + queue — but theirs is a deployed service with history, versioning, and no key exposure, built years earlier.

### 4. The strategy layer

Four patterns recur everywhere in their repo and deserve names:

1. **Expected-gain-proportional attention**. The v2 scheduler samples the next problem to work on with weight `2 - Unagi's current points` on that problem (2.0 being the per-problem maximum). Modes `improve` and `rewrite` are chosen 50/50; the target is a randomly chosen team *above* them (or their own best when ranked 1). The explore half — `rewrite` — is what produced the  $\geq 100\times$  jumps.
2. **Floors first, ceilings second**. For llm they shipped a public-case memorizer (14/28 cases, honest filename `14-of-28`) as a floor while the real universal interpreter was being built. Cases-passed is half the points; a floor is nearly free.
3. **Leaderboard-coupled everything**. Postmortems quote the leader's score and the gap ("gap to leader 6.8%"); one artifact records the policy "commit/push only after reaching rank 1"; the v3 agent loop polls standings before every run and *sleeps while rank 1*.
4. **Everything regenerable, everything scored**. Artifacts are named by official score; generators + seeds are committed next to outputs and byte-reproduce them; placements ship PNG renders and JSON routing reports; experiments that FAILED are written down with numbers (memory's 2-worker rotation: "no average gain, heavy consecutive-access cases regress" — recorded, rejected).

### 5. The agent systems: v1 → v2 → v3

Three generations inside one 72-hour contest:

- **v1** (from 17:32 UTC day 1): flat timestamped work dirs, one-shot Codex runs per problem.
- **v2**: three roles. *Solvers* (agent sessions humans launch in tmux) ask a *Scheduler* for work — an SQLite service holding free-form `suggestions` (FIFO, from humans or the Curator) and falling back to the expected-gain sampling above; a *Curator*

— deliberately a **single writer** — wakes every 300 s, reads new solver memos, integrates them into per-problem memory and a cross-problem Cookbook, and posts cross-pollination suggestions back to the scheduler.

- **v3:** the mature loop ( `loop.sh <slug>`, `loop_v2.py` ). Each run launches a **fresh-context agent** that inherits the durable tree: `problems/<slug>/` (current position + solution history), `knowledge/` (cross-problem techniques), `work/` (scratch). The solver prompt is a six-step pipeline with concrete shell per step — assess (auth, ff-only pull, resolve the problem via the public API) → solve → submit → record → notify → save. `loop_v2.py` fetches standings before each run and sets the success criterion dynamically: catch rank 1; if more than 10× behind, achieve ~10×; if rank 1, sleep 10 minutes and re-check. A `semantic-regressions.json` records meaning-level mistakes past agents made, as a regression suite for prompts.
- **v4-hand:** the human pair — wata + iwiwi trading artifacts on the hardest interpreter problems (LLLM’s descent 3.8T → 68.97B is a filename trail alternating `_iwiwi` / `_wata-orz`), with detailed architecture notes ( `LLLM_BITPLANE_PIPELINE.md` etc.).

Contrast with ours: we ran 4–8 heterogeneous agents (claude, codex, gpt, chatgpt\_1/2/4, codex\_3, alexey) coordinated by a human + one coordinator agent over git-messages, with manual task (re)assignment. Our topology excelled at **handoffs** — ten of our fourteen final-day improvements were other agents’ finished work that only needed judging and pushing. Theirs excelled at **unattended grinding against the leaderboard** — fresh contexts, durable knowledge, machine-checkable goals. The two models are complementary, and neither team ran the other’s.

# Part II — The Machinery Behind Each Stage

---

Their pipeline, stage by stage, with the exact artifact that implements each. The through-line: **at every stage, the unit of work is a regenerable, testable, scored artifact — never a hand-edited grid.**

## Stage 0: Ground truth — `unagi-simulator` + conformance

~10k lines of Rust. Values are `i64` with the official wrapping semantics; the mid-contest `Y` (split) instruction is implemented; program boards are flat row-major byte arrays.

The decisive design choice is *dual execution*: `./bin/simulate` runs the Rust engine, `./bin/simulate-official` runs the organizers' own Go/WASM build behind the same CLI, and both consume a shared conformance file (`crates/simulator/testdata/cases.json`). Divergences between spec and official behavior are cataloged in `docs/simulator/language-conformance.md`, and recorded probes of the live editor sit in `docs/official/editor-observations-2026-07-26.json`. Solution harnesses routinely ship **both** verifiers (snake: `verify.rs` and `verify-official.mjs`; llm cross-checks its pipe shifter and guest ALU against the WASM).

Ours, for contrast: `sim.py` + C extension + Rust executor, with the official WASM added as a judge only on day 3 — after an unimplemented `Y` and a wrong wall rule had each cost us a submission. Same final architecture; theirs was doctrine from hour one, ours a repair.

## Stage 1: Algorithm selection — humans, with a rewrite budget

No tool chooses algorithms; people do, and the operation *forces the question* twice: the scheduler's 50/50 improve/rewrite modes, and postmortem templates that open with “computation graph / pseudocode” before any layout detail. Algorithm changes are tracked as first-class events: pathfinder has three distinct generations in its artifact trail, matmul two, subset-sum two.

## Stage 2: Composition — gadgets with contracts

Two implementations of the same discipline:

- `crates/hand` (~9.9k lines): gadgets as Rust **async actors** — typed ports over channels (`OutPort / InPort`), a room's behavior unit-tested against its boundary trace via `assert_room` (with tick limits), `auto_layout` for placement, a display model. Entire solutions are emitted by binaries (`cargo run -p hand --bin subsetsum2 -- --layout out.man`) and byte-compared against the committed artifact. The regression tests encode their scars: false pipe connections at wall-adjacent bends; collisions after the final output.
- `chokudai/compiler v0`: a hardware-CAD flow. Input: a flat physical netlist of gadget instances + port connections. Passes: port/protocol validation → dependency-graph **layer placement** → *A pipe routing with fixed terminal flow → nearest-port contract check\** (does every `s / r / q` still bind to the pipe the netlist meant?) → emit `.man`. Implicit fanout is rejected. The gadget catalog (`chokudai/gadgets/`) holds verified parts — crossovers, persistent counted adders/mergers, phase gates, relays, a matmul PE — each with port order, backpressure and tick behavior recorded.

chokudai/README.md states the doctrine that connects them: artisan small problems → harvest verified gadgets → compose large problems from gadgets via generators → fuse hot combinations back into larger artisan macros → reuse. Decide per problem: one dominant computation → artisan; several independently verifiable subsystems → compose.

## Stage 3: Generation — the per-solution compilers

Every serious artifact ships its generator: `<score>.py` (or `.rs`) plus, on the biggest machines, `_router.py`, `_compact.py`, `_finalize.py`, a placement PNG, and a JSON routing report. Generators own the problems we fought by hand: literal packing (history-lesson’s generator verifies every 64-bit literal and aligns widths so **no accidental vertical backtick pair** can form), nearest-pipe preservation (matmul’s router validates **40 nearest-pipe sites** against the donor: “the nearest-pipe identity as seen from each room is part of the ABI”), routing search (snake’s router records `seed 364` — routing is a searched, reproducible artifact).

The postmortems repeatedly separate “computation graph identical; layout only” steps from algorithm steps — the two are different artifacts with different verification obligations.

## Stage 4: Optimization — `unagi-optimizer` (+ `optimizer_cli`, ~21.8k lines)

`optimize <slug> <valid.man> --time-limit --seed [--submit]` — local search over an already-valid program:

```
Solution (.man) → Program → Layout (rooms/pipes, read-only view)
      ↑                               ↓
accept/reject ← EvaluationBackend ← Modifier::propose
      ←———— Search (annealing/greedy) —————→
```

- **Modifiers** (plugin trait, stable IDs): `trim-empty`, `room-shrink`, `room-move`, `pipe-reroute`, `relocate`, `room-deadcell`, `room-op-slide`, `room-commute-swap`, `room-route-nudge`, `room-corridor-reroute`, `room-op-corridor-reroute` (carries pure register ops along a rerouted corridor while preserving heading sequence and visited-cell count; `s / r / q` deferred until nearest-pipe-invariance checks land).
- **Safety model**: every modifier is tagged `changes_timing()`; a differential **equivalence gate** (`--equiv-gate`) is enforced for timing-changing modifiers only — their systematization of the “pipe length is latency AND capacity; never shorten a storage pipe” trap that we learned by deadlock.
- **Evaluation**: public cases in parallel (Rayon), result order stable; the search core is pure (injected clock, event stream) so it tests without files or wall time.
- **Runner**: progress events, best-artifact saving, submission with backoff — it can run unattended and submit its own improvements.
- The **roadmap document** ranks candidate modifiers by measured criteria: leverage (footprint =  $\max(w,h)^2 \Rightarrow$  only a fully cleared perimeter band pays; neutral moves are still valuable as enablers), valid-yield, firing frequency (measured per problem: one modifier fired zero times on snake’s best — deprioritized), timing class. An interior survey quantifies the opportunity: best answers are “extremely roomy” — snake ~11.4% dense, gradebook ~6.8%, with the blank space *inside* one giant room.

Measured contribution on visible artifacts: pathfinder 81.7B → 80.9B, matmul 500M → 495.8M (~1% each), with better-scored optimizer outputs kept as candidates for gradebook. The pattern: **generators and algorithm changes earn the orders of magnitude; the optimizer converts idle CPU into steady percent-scale gains.**



Ours, for contrast: this tool is nearly clause-for-clause the “manifest” our team dictated mid-contest (a randomized, high-performance Rust room optimizer applying semantics-preserving transformations). They had it running by 07:43 UTC on day 3; we wrote the idea down at roughly that hour and built the analogous machinery (room contracts, then post-contest `room_ir / room_place / room_compile`) after the end.

## Stage 5: Reading machines — `crates/mandis`

A disassembler/decompiler: validated `.man` → Rust-like pseudocode with `goto` labels, room/pipe numbering, source coordinates, and **static per-block tick counts**. MVP by their own description, but it closes the loop: machines are readable, so foreign or old artifacts can be understood, priced, and re-entered into the generator flow. (Our `room_ir` is the same idea pointed at re-emission.)

## Stage 6: Scoring, submission, records

- `crates/scorer` — the official formula over public tests (`./bin/score --offline --compact`), used as the acceptance gate in every postmortem.
- The submission relay (Part I §3): queue, versioned storage, event history, single holder of the API key; plus `crates/submit`’s `scoreboard` binary re-scoring everything like-for-like.
- Every postmortem records: result, local + official submission IDs, SHA-256 of the artifact, geometry, tick statistics, what changed versus the donor, and a verification checklist.

## Stage 7: Knowledge — catalog, textbook, editorial

- `docs/techniques/README.md` — a 56-entry reverse-lookup catalog (“what do I reach for?”), each entry: one-line summary, mechanism, cost/benefit, named example solutions. Its preamble is the scoring physics (judge by footprint<sup>2</sup> × ticks product; footprint-only problems excepted).
- `docs/textbook/` — the same knowledge re-indexed by goal (part 3 techniques, part 4 construction patterns), 1:1 links to the catalog.
- `docs/official/` — bilingual mirrors of every official document, plus recorded editor-behavior probes.
- `iwiwi/editorial/` — a post-contest interactive site (Next.js): each explanation descends algorithm → computation graph → pseudocode → *the real manifest*, with hover-highlighted regions and “guided gadget execution” that steps a chosen gadget with live room state. Explanation is treated as a first-class engineering artifact with its own authoring guide.

A selection of catalog entries that name things we met the hard way: `#17 Geometry Is the Operand` (nearest-pipe binding), `#30 Fixed-Width Aligned Literals` (vertical backtick control), `#34 Keep Scored Snapshots`, `#38 Phase Capacity of a Shared Loop` (`lap/gcd(lap,T) ≥ N`), `#42 Latency-Preserving Fold`, `#44 Cursor-Relative Ring`, `#47 Epoch Tag Instead of Clearing`, `#56 Differential Dynamic Profiling`. And entries we never had: `#36 Y Fan-Out`, `#37 Y Delay-Line Sort`, `#48 Temporal Register Aliasing`, `#52 Multiply-Then-Shift Classifier`.

## Part III — Problem by Problem

---

Sixteen graded problems. For each: what the problem asks, Unagi’s algorithm and machine, how the artifact was produced and verified, the score trail, and the comparison with our solution. Scores are official unless marked *public*. “Snapshot” = 2026-07-27 08:38 UTC standings.

---

### pathfinder — 141× (their 114B vs our 16.1T at snapshot)

**Problem.** A 16×16 maze arrives as 256 wall bits plus a robot start; flags appear one per round; guide the robot to each flag and draw its path on a 16×16 display.

**Their trail — three architectures in 48 hours:**

*Generation 1* — “four-lane bit-wave BFS” (80.9T, 18/18, 3783×3812, 9.0 MB of text). Algorithmically already advanced — walls packed into sixteen 16-bit row masks, reverse BFS from each goal by bitwise wave propagation in four parallel row lanes, parent directions recorded, a streaming walker replays the path to the display. But the machine was enormous; at box 3812 the footprint term buried it.

*Generation 2* — “16-row parallel BFS” (62.2T, 3456×1077), generated by a Rust program ( 62166740306242.67.rs ) that emits basic blocks, branches, a local router — and a **bus protocol**: the CPU sends `target`, `action`, `payload` triples over shared pipes; every room self-selects by target ID (entry sequence `rM8W-` tests “is this for target 8?” in five cells); row actors answer a probe with the single value `2*(16*y+x) + is_open`, folding the branch result and coordinates into one integer so the CPU never re-reads metadata per direction. Board rows are 64-bit words at 4 bits/cell (wall, visited, 2-bit predecessor); probes go in fixed  $U \rightarrow R \rightarrow D \rightarrow L$  priority.

*Generation 3* — “packed row-bitset reverse BFS” (681B → 211B → 141.7B → 133.7B → 132.7B → 121.7B; sides 238 → 231 → 222). The redesign that mattered. One 64-bit word per maze row:

```
state[row] = available | parent_low<<16 | parent_high<<32 | frontier<<48
```

A 16-word ring holds immutable free-cell masks and resets the dynamic words each round (epoch-style, no clearing pass). The BFS expands the frontier by whole-row bit operations and scans **only the row span the frontier occupies**,  $\pm 1$  — U/R/D/L candidate masks consume `available` in priority order so equal-distance ties resolve deterministically. During reconstruction, robot and flag positions are packed into one integer and arrival is tested modulo 65537. Machine  $\sim 222 \times 222$ , official average  $\sim 1.78\text{M}$  ticks. Each step ships generator + finalizer + router scripts and a placement PNG; the final 5.65% came from a layout-only change (a 9-register bank moved 2 cells left, “instruction streams byte-identical, all 38 pipe endpoints preserved”). Their optimizer adds a final  $\sim 1\%$  (81.7B → 80.9B artifacts).

**Ours.** A per-cell walking BFS in a 181×936 room (the machine’s whole height); contest improvements were geometric (3.0× then 1.227× from snake line-merges) landing at 4.37T. Our post-contest measurement explains the residual: 90.8% of our room-0 walk was inter-operation transport under nearest-pipe lane structure — but even a perfect re-layout of our machine cannot reach their numbers, because their frontier step is  $O(\text{rows})$ , ours  $O(\text{cells})$ . The gap decomposes roughly as: algorithm  $\sim 40\times$ , machine size  $\sim 4\times$ .

---

## matmul — 356× (16.7M vs 5.93B)

**Problem.** Multiply an  $N \times M$  by an  $M \times K$  matrix (dimensions bounded by 16), all values streamed.

**Their trail.** Early: a generated 16×16 processing-element grid (kept as `answer.man` + generator, marked legacy) — scores descending 685M → 446M. Then the REDESIGN: a **scalar ring-stream** —

```
for i in 0..N: sums[0..K] = 0
  for t in 0..M: for j in 0..K: sums[j] += A[i,t] * B[t,j]
  emit sums[0..K]
```

realized as: A expanded elementwise  $\times K$  in  $(i, t, j)$  order; B circulating row-major in a pipe ring; the K partial sums circulating in a FIFO whose storage is an **exact 261-cell pipe** — the pipe's length is simultaneously its capacity and the loop's phase; a row-marker stream (`M-1` zeros then 1) delimits rows. Ten rooms, fifteen pipes. This dropped them 446M → 96M → 76M → 36.8M (50×47).

The final 15% (36.8M → **31.1M**, 46×46) is a pure placement compaction with the topology frozen: a Rust placement generator, four left-boundary crop passes, one local room move, and a router that re-verifies **40 nearest-pipe sites** against the donor — their postmortem's phrase: "the nearest-pipe identity as seen from each room is part of the ABI." The input pipe alone was shortened 10 → 2 cells (−8 ticks average); every other exact pipe length preserved.

**Verification.** Exhaustive: all 3,375 dimension triples, 20,250 boundary-value cases, 4,096 random, 23 targeted; byte-reproduction of the artifact; the harness committed.

**Ours.** 5.93B from a hand-architecture with a 1.42× squeeze — we never revisited the algorithm. Both of their jumps were closed to us: the ring-stream (algorithm) and the frozen-topology compaction (we had no re-placer; our post-contest compiler is exactly that tool).

---

## subset-sum — 37,601× (995M vs 37.4T)

**Problem.** Choose a subset of  $\leq 20$  integers summing to a target; output the count and the chosen values in original order; lexicographically smallest index set among solutions.

**Their algorithm — meet-in-the-middle, in pipes.** Split indices into even/odd halves ( $\leq 10$  each  $\Rightarrow \leq 1024$  subsets per half). Each subset is ONE 64-bit record: subset sum in the high bits; the low 40 bits interleave a lexicographic priority (index  $i$  at bit `39-i`) and the selection mask (index  $i$  at bit  $i$ ) — constructed so that among equal sums, the numerically largest record IS the lexicographically smallest index set. Ordering logic becomes plain integer comparison.

The two halves' subset lists are generated as **descending sequences merged in pipe loops**: `duplicate` and `merge` rooms merge "current sequence" with "sequence + next contribution" on the fly (left branch adds positive contributions; right branch subtracts from the target). `decode` unpacks; `match` two-pointer-walks the two descending streams; `max` keeps the best matched record; `select` replays the archived input values against the selection bits, emitting count + values in original order.

**The machine.** 106×106, 17 rooms, 25 pipes; the MITM lists live in **snake pipes with exact capacities** — left-a0 529 cells, left-a1 517, right-a1 540, right-a0 512, all ≥ the required 512 records, one snake detouring around a room “to make up missing capacity”. Central pipe bands are documented column by column. Generated by `crates/hand (--bin subsetsum2)`, byte-compared; 13 unit tests + 25 `hand` library tests; each public case run 118 ticks past the final output to prove no post-answer wall collision. They also submitted a trimmed and an untrimmed (blank-margined) version and observed the SAME official score — establishing that the scorer normalizes surrounding whitespace (an experiment, recorded).

**Trail.** 10.4B → 9.2B → 9.0B → 690M → 669M → 531M → **443M** (their repo best; snapshot 995M was mid-descent). 31 postmortems — their most-iterated problem.

**Ours.** Direct enumeration; 37.4T. No layout work recovers a complexity-class difference; this single problem is ~1 point of standing.

---

## sudoku-validity — 3,709× (2.50M vs 9.29B)

**Problem.** Validate 9×9 Sudoku boards streamed cell by cell; emit 1/0 per board.

**Their machine — 28×28, branchless.** All 27 constraint groups (9 rows, 9 cols, 9 boxes) are 9-bit used-masks packed into FOUR signed 64-bit words, with each group’s field placed so the word’s **sign bit stays free** (word 0: rows 0–6; word 1: cols 0–4 + rows 7–8; word 2: cols 5–8 + top boxes; word 3: middle + bottom boxes). An input digit *v* becomes `1 << (v-1)` shifted into its group’s field.

Four holder rooms sit in the corners, each looping  $R \sim W - s$ : receive candidate, `new = old XOR candidate`, store, reply `old - new` — which is **negative iff the bit was previously unused**. The central aggregator ANDs the row/col/box replies: all-negative ⇒ still valid. Aggregation gates the next candidate broadcast, so state updates can never be overtaken — lockstep by protocol, not by timing luck.

Layout notes show nearest-pipe-aware micro-routing: a reply path was shortened 17 → 15 cells, and the postmortem explains why it could not be shortened further — the next bend would put a generator’s `s` closer to the wrong pipe. Verified with 527 differential tests (27 single-duplicate archetypes + 500 deterministic-random boards) against a reference implementation.

**Trail.** 2.88M → 2.86M → 2.73M → 2.63M → **2.50M**.

**Ours.** 9.29B (later 6.74B): a sequential checker walking per-cell logic. Their entire validation runs in ~3,153 average ticks on a 28×28 footprint; ours is both larger and thousands of times slower — pure algorithm (bit-packing + sign-classification).

---

## plotter — 271× (3.72M vs 1.01B)

**Problem.** Execute drawing commands (pen moves/lines) on the LM-75 display; scored per round on emitted frames.

**Their machine — 50×50, 13 rooms.** Architecture held fixed from the F=58 generation; two engineering moves did the rest:

1. **Literal → arithmetic so code can bend.** A backtick literal is straight-line by definition — it cannot turn a corner. Replacing ``32`` with `5M1{` (set 5, shift 1 by it) and ``4096`` with `3M4*M` makes the instruction stream foldable, at which point

every room was **comb-folded** (“fold4”: a  $w \times 4$  box yields  $4w-8$  op slots vs a ring’s  $2W+2H-8$ ). Thirteen rooms re-folded with before/after tables (Iterator  $19 \times 7 \rightarrow 9 \times 7$  with the 8/10-cell hot loop untouched — only cold init/teardown paths were folded; Writer  $15 \times 6 \rightarrow 10 \times 6$ ; etc.).

2. **“Plan R”: the floor as a planar graph.** They stated the invariant first — a horizontal pipe must cut vertical channels — then chose the ROOM ORDER so that exactly one net ( `pYM` ) crosses the north-south divide, and 19 pipes routed on the first attempt. The previous generation’s blocking crossing ( $D \rightarrow Y$  over  $K \rightarrow F$ ) was removed by reordering rooms, not by cleverer routing. All pipe lengths are listed in the postmortem; six shrank dramatically ( `pDV` 69  $\rightarrow$  3).

Underneath sits a measured cost model: **round ticks** =  $142 + 8.4 \times \text{pixels}$  (previous generation:  $156 + 8.5p$ ), so footprint  $58 \rightarrow 50$  ( $58^2/50^2 = 1.35\times$ ) was correctly identified as the dominant term. Then they measured **per-pipe criticality empirically** — lengthen each pipe by +8 cells, observe the public-average delta — producing a tick/cell price list (`pDX` 3.5/cell ... `pVF` 0, non-critical), and noted the two initialization critical paths are balanced (X branch 40 vs Y branch 42): “max applies — shortening only one is wasted.” A `redesign/DESIGN.md` carries the timing constraints checked at generation time.

**Trail.** 23.96M  $\rightarrow$  23.92M  $\rightarrow$  5.46M (F=58)  $\rightarrow$  5.05M  $\rightarrow$  **3.87M** (F=50); they logged the then-leader (3.627M) and their 6.8% gap.

**Ours.** 1.01B (958M final) from a hand machine + moves; our plotter notes measured the same phenomenon (1,642 blank runs; delay realized as distance) but the counted-loop fix stayed unbuilt.

---

## gradebook — 116 $\times$ at snapshot (405M vs 47.1B; both improved later)

**Problem.** Maintain a roster of  $N$  students  $\times$   $K$  grades under batches of GET/SET/AVG/TOP operations.

**Their machine — 83 $\times$ 89, a circulating roster.** The roster is a fixed-width FIFO ring `[-N, id, g1..g4]  $\times$  N` (grades zero-padded to width 4); the negative record `-N` is both the count and the rotation marker, and **every operation rotates the ring back to the marker**, so each op starts aligned at a record boundary. GET/SET rotate and read/replace; AVG streams the selected grade column into an accumulator and emits `floor(sum/N)`; TOP streams (grade, id) pairs through a GradeComparator  $\rightarrow$  MinimumIdKeeper (a persistent compare-and-keep actor — catalog #26).

The committed improvement is pure packaging: TOP/AVG support actors replaced by smaller tiles, then **coordinate compression** — empty columns inside the main room and empty rows/columns globally deleted one at a time, “re-running the real simulator to 7/7 after every deletion” —  $92 \times 94 \rightarrow 83 \times 89$ . The artifact ships a regeneration script plus donor manifest. Optimizer artifacts dated 07-26 07:43/07:55 UTC carry 167.9M — the unattended polisher finding further compression on the same architecture.

**Ours.** Our gradebook win was the same *family* of insight — a result-collector confirmation pipe cutting ticks  $95,493 \rightarrow 73,931$  ( $1.355\times$ ) — but executed on a 379-wide machine; theirs is  $\sim 4\times$  smaller in footprint with  $\sim 5\times$  fewer ticks. Catalog cross-links: #42 latency-preserving fold, #43 semantic NOP blanking, #56 differential dynamic profiling (find hot instructions by diffing two runs with different op counts).

## Part III (continued) — The Interpreter Family and Snake

---

The three “little man” self-hosting problems asked teams to build littleman machines that *execute littleman programs*. These reward interpreter architecture above all — dispatch cost, memory representation, display protocol — and they are where Unagi’s human-pair (wata + iwiwi) worked deepest.

---

### little-little-man (llm) — 545× (16.1T vs 8.78Q)

**Problem.** Execute an arbitrary guest littleman program (16×16 room, up to 3 men, pipes, full instruction set) and render each tick to the LM-75 display. 28 cases: 14 public, 14 private.

**Solution A — the strategic floor: a public-case memorizer.** A Rust generator ( `14-of-28.rs` ) emits a 282×274 machine that *recognizes* which public case it is running (broadcaster → 14 case-source rooms matching W, H and one identifying cell) and replays embedded frame diffs through a rasterizer + display adapter. Result: public 14/14, private 0/14 → **14/28 cases**, score `null` (no footprint-tick score without a full pass), filed under the honest name `14-of-28`. Under points = cases-fraction + rank-fraction, half the cases is a real floor, shipped while the real interpreter was built. They verified all 135 embedded frames against official data.

**Solution B — the universal banked interpreter (958×713, 28/28, 16.09T).** A littleman machine emulating a littleman machine:

- the guest’s 16×16 grid lives in a **256-cell circulating memory**;
- guest room/man/pipe metadata occupy three 64-cell circulating memories plus **39 small scalar holder rooms**;
- a central controller executes up to 3 guest men in **lockstep**, implementing `^ > v <`, digits, `M + - X s r H` with signed-64 semantics;
- guest pipes update tail-first, reproducing the official same-tick cascade; send/receive blocking, nearest-endpoint selection and reading-order tie-breaks are **computed at runtime from coordinates** — the interpreter re-derives the binding rules we all fought, per guest step;
- initialization paints the full frame; each round commits exactly one frame of diffs.

The geometry work is its own story: the controller’s command fan to ~50 service rooms was compressed from 180 columns to 152 by shifting the service column 15 rows and re-grading every command pipe’s slope so none cross; reply pipes drop into 2-row gaps under each service and merge into a 7-column vertical collector (replacing 50 independent return tracks); the 256-cell ring folds to 29×16. Verified by unit gadget tests plus **pipe-shifter and guest-ALU cross-checks against the official WASM**; official average ~15.9M ticks.

**Ours.** Also an interpreter (28/28) — at 8.78 QUADRILLION. The 545× is pure constant factor: our guest-step dispatch and memory walks cost orders of magnitude more ticks, and our machine was larger. Their 39 scalar holders + banked rings + lockstep controller is a articulated answer to exactly the constant-factor question.

---

### little-little-little-man (lllm) — 4.4× (4.81B vs 21.2B)

**Problem.** A reduced interpreter-of-interpreter task (guest programs from a constrained subset; 21 cases).

**Their trail.** The LLLM/ directory preserves a hill-climb as `<uuid>_<score>_<author>.man` artifacts alternating between iwiwi and wata: 3.822T → 423B → 187B → 103.7B → 75.98B → 69.1B → 68.97B; the final redesign lands in `little-little-little-man/` at **11.35B (21/21, 130×172, official avg 383,637 ticks)** — and the snapshot shows 4.81B, i.e. they kept descending after the last committed postmortem.

**Final architecture — “direct scalar service + pipelined controller”.** The guest program is stored as **5-bit opcodes in a 260-element cursor-relative FIFO** (catalog #44: rotate a logical head pointer, never move the data); execution state sits in an 8-element direct scalar service; the controller talks to exactly four subsystems: program memory, scalars, input, display writer. The final committed 8.77% came from packaging: the init loop’s 15 recurring “program request → display DATA” pairs laid out on ONE row, the remaining control flow shelf-packed beneath, then **49 empty controller rows deleted**.

**Ours.** 21.2B at snapshot, 17.6B final — our strongest showing among the interpreter problems (a subagent’s eight-row shrink 303×311 → 303×303 with the merge+rail technique). Their advantage here was sustained human-pair iteration; the architectures rhyme (both ring memories with cursor state), and this is the problem family where our approach came closest on merit.

---

## snake — 7.6× (107M vs 809M)

**Problem.** Play snake on a 16×16 board: fruit spawns, direction changes arrive, body grows; draw every state; detect self-collision and wall exit.

**Their machine — 85×85.** The snake body IS a **physical FIFO of cell addresses**, tail → head. Five scalar “echo holder” rooms keep `dir / fruit / head / len / c16`; the controller keeps the candidate head in register B and the remaining scan count in the **backpack**; self-collision = one full lap of the body FIFO comparing addresses. Drawing is incremental (catalog #54): head pixel + tail pixel per move with `SWAP 1` preserve, plus fruit on spawn; round init draws once.

**Toolchain per artifact** (the fullest example in the repo): `generate-up.py` (generator) + `router_hotloop.py` with recorded **seed 364** (routing is a searched, reproducible artifact) + `verify.rs` AND `verify-official.mjs` (both engines) + `generate-random.py` / `generate-stress.py` (fuzzers) + `final.png` (placement render). The committed best (234M) documents shrinking the Incremental Writer 13 → 10 columns and the controller’s bottom margin 6 → 2 rows, re-routing in the narrower room with two manual control-edge reroutes; optimizer artifacts carry 157M; snapshot 107M.

**Ours.** 809M — a 1.05× line-merge on an inherited machine. Theirs is what the same problem looks like when the machine is generated: every shrink is a re-run, not a surgery.



## Part III (continued) — The Small-Machine Problems

---

Where machines fit in tens of cells per side, the competition is protocol design and single-cell placement. These are the problems where our numbers were closest to theirs — and once, better.

---

### brackets — 4.5× (83.8K vs 377K)

**Problem.** Stream a bracket string; output 0 if balanced, else the 1-based position of the first error.

**Their machine — 16×16, four stages.**

1. *Feeder* (lower-left): drops the length header, forwards each byte, appends a 0 end-marker.
2. *Classifier* (upper-left): computes `(5 × ASCII) >> 5` — one multiply and one out-of-range-safe shift map all six bracket characters onto three type codes {6, 14, 19} (catalog #52), with closers negated.
3. *Worker* (lower-right): the WHOLE stack is one base-3 number in the off hand (catalog #7): `push = stack*3 + type ; pop = subtract the type and divide by 3` — a nonzero remainder is a type mismatch, a negative quotient is a pop-on-empty (catalog #8: divide is divmod).
4. *Counter* (upper-right): counts accepted characters; converts the first negative status into the answer position; end-of-stream with zero status emits 0.

Micro-placement: the stack room was widened by one cell to shorten the pop loop from 18 to 16 ticks; to keep the 16×16 footprint the input room moved to the corner behind a 3-cell bent pipe. Official 26/26, average 184.1 public ticks.

**Ours.** The 22×22 line (377K, landed 1.286×) — we fought at footprint 22 vs 23 while they sat at 16 with a register-packed stack. The base-3-stack-in-a-register idea removes the storage pipes that forced our size.

---

### reverse-a-list — 4.2× (20.0K vs 84.4K)

**Problem.** Read N values per round; emit them reversed.

**Their committed prototype of note — the Y delay-line** (catalog #37): one dispatcher man per round Y-splits per value; the east-born child parks the value in a **private counted loop** (8 ticks per remaining item, counted in BP), the west-born child returns in 6 ticks to split again; later values therefore finish waiting *first*, and the single output pipe receives the list already reversed — no per-value rooms, no collector network. Their prototype measured 130.0 public ticks against their then-best 171.6 but stood 22×69 (“too tall, needs folding”) — committed as a design with a note, alongside U-demux/token-handshake solutions in the trail (110K → 100K committed; a mid-trail note records official 47,059 and the policy “push only after reaching rank 1”; snapshot 20,026).

**The convergence worth recording:** our teammate gpt built the same Y-farm architecture during the contest and proved it correct, but its delays were SPATIAL (worker *i* waits by walking,  $\Sigma(D-2i) = W^2$  cells) and it could not fit the winning box. Our own `claude_42` note identified the repair — “counted loops instead of distance” — hours before the end. Unagi’s prototype is that exact repair, implemented: delay in TIME (backpack counter), not space.



**Ours.** 84.4K at snapshot via chatgpt\_1's landed 1.349×

---

## tcp — 1.7× (821K vs 1.36M) — our closest engineered gap

**Problem.** Reassemble a packet stream: emit values in sequence order, -1 for each packet outside the 16-wide window.

**Their machine** — 16×17. Future packets as (seq, val) pairs in a circulating FIFO; next-needed sequence in register B. The out-of-window test is branch-free: copy `seq - next` to the backpack and right-shift by 4 — any nonzero result means  $\geq 16$ , emit -1 (catalog #28 + #9). Scan sentinel is 0 (never a valid sequence number). The capacity ledger is explicit and exact: 2 cells outbound + a 28-cell folded return + **1 integer held in the forwarder's hand** = 31 integers = 15 packets × 2 + sentinel — registers and pipe cells budgeted as one storage pool. The main room's comparison/reinsertion column was moved left (11 → 10 columns) by resetting the backpack via the comparison's own 0 result and re-saving through an edge `b` — same state machine, one column thinner. Verified with 128 random valid + 64 loss cases.

**Ours.** 1.36M — codex's tcp line with the router optimization; our closest gap among seriously-engineered problems, and the clearest case of “same idea family, one protocol trick apart.”

---

## memory — 1.6× (8.85M vs 14.0M)

**Problem.** A random-access memory service over pipe storage.

**Their trail.** Iterative packing with recorded numbers: wip 11.43M → 9.49M “packed to 22×22” → a 21×21 note; and a REJECTED experiment documented with its measurement — parallelizing the FIFO rotation with 2 workers (split  $n/2$ ,  $(n-1)/2$  so the second skips the loop when negative) targeting 4 ticks/item: no average gain, because consecutive-access-heavy cases regress. Recorded, then abandoned.

**Ours.** 14.0M, including the user's own hand-edit (12 cells moved in the visual editor, -2.71% ticks) — the single event that seeded our whole “optimize rooms preserving function” manifest. This is the problem where our *method* (careful local moves under contracts) most resembled theirs; they simply ran more iterations of it.

---

## sort-numbers — 2.5× (320K vs 802K)

**Problem.** Sort each round's values ascending.

**Their machine** — 13×13 **selection-sort ring**. Each pass carries the remaining count as a header (catalog #10 — no sentinel value stolen from the domain); the candidate minimum rides in B; `X` sign-branches on the difference; non-candidates recirculate. Duplicates and the domain edges  $\pm 10000$  need no special cases. The committed micro-step: bend the zero (equal) branch WEST into the same `+` that the positive path uses to restore `x`, then a dedicated adjacent `s`, rejoining the shared scan-count decrement — 4 ticks saved per equal comparison, footprint unchanged. Verified with a 200-round, 1,716-element multi-round session on both engines. Committed 377K; snapshot 320K.

**Ours.** 802K (1.012× late gain). Same algorithm family; theirs is one protocol generation tighter.

---

## history-lesson — the problem we won (their 6,889 vs our 6,561)

**Problem.** Output a fixed 2,810-byte ASCII text (footprint-only scoring — ticks don't matter).

**Their machine — 83×82.** The text compiled to 3,020 base-34 tokens (lowercase alphabet + 2 escapes + 5 frequent phrases); tokens packed 10–12 per signed 64-bit backtick literal; the GENERATOR validates every packed integer fits 64 bits read from either direction and lays 4 integers per two-row group with **literal widths matched so no accidental vertical backtick pair can form** — generation-time control of the column-pairing rule. A three-stage decoder (base-34 remainders → escape/phrase expansion → ASCII) sits beneath the data block; two decoder rooms were tucked beside an existing 2-cell pipe to shave the height 84 → 82.

**Ours — 6,561, rank 21 vs their 28.** Our history tokenizer line (phrase dictionary + packed literals, `claude_35`) found a slightly better encoding/geometry product. On the one problem scored purely on the axis our tooling was strongest at — literal packing under the backtick rules — we beat them.

---

## triangle — parity (832 = 832)

The warm-up (triangular numbers). Both teams at the shared optimum 832; rank 59 vs 118 is submission-time tie-breaking. Unagi kept no solutions directory for it.

# Part IV — The Two Teams, Side by Side

## 1. Structural comparison

| dimension                       | Unagi                                                                                                                  | wheezards                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| people                          | 6 veteran humans + LLM agents as workers                                                                               | 1 human + 5–8 LLM agents; one agent as coordinator                                                       |
| infrastructure                  | ported from 3 prior years, live before start (fleet executor, Cloud SQL, dashboard, submission relay)                  | built during the contest (judge scripts, submissions DB, board reader)                                   |
| simulator                       | Rust, Y included, conformance suite vs official WASM from the start                                                    | Python + C ext + Rust, WASM gate added day 3 after two paid divergences                                  |
| how machines are made           | per-problem generators (Python/Rust) over gadget libraries; hand only at the frontier                                  | hand/agent-written grids, then surgical edits                                                            |
| optimization                    | generic annealer over 11 semantics-preserving modifiers, equivalence-gated, auto-submitting                            | problem-specific squeezers + hand edits; the generic optimizer existed as a manifest, built post-contest |
| algorithm changes               | forced by process (rewrite mode 50/50; postmortems open with the computation graph)                                    | rare; mostly inherited first working algorithm                                                           |
| verification                    | per-solution checklists, exhaustive/fuzz harnesses, both engines, byte-reproducible generators, SHA-256 in postmortems | preflight + subdb like-for-like judge; strong late, absent early                                         |
| knowledge                       | 56-technique catalog + textbook + ~230 postmortems + bilingual official docs                                           | 45 claude_NN analysis docs + handoff messages; analysis-shaped, not retrieval-shaped                     |
| submission                      | Cloud Run relay, queue, versioning, key isolation                                                                      | single gatekeeper agent with local key                                                                   |
| agent topology                  | scheduled fresh-context loops vs the leaderboard; curator as single knowledge writer                                   | persistent heterogeneous agents with human-mediated handoffs                                             |
| final standing (08:38 snapshot) | 11th, 29.31                                                                                                            | 39th, 24.68                                                                                              |

## 2. The convergences — same physics, different clocks

Reading their repo after building ours is an exercise in recognition. Independently, both teams derived:

- **The scoring calculus:** judge by footprint<sup>2</sup> × ticks product; only a fully cleared perimeter band changes the footprint term (their optimizer roadmap; our claude\_44).
- **The four ways a moved cell breaks:** heading, nearest-pipe binding, timing/phase, literal pairing (their in-room rewrite semantics reference; our room\_reflow/room\_shrink checks).

- **Geometry as ABI:** their matmul router verifies 40 nearest-pipe sites; our room\_ports/binding\_map machinery does the same check.
- **Roominess:** their survey (snake 11.4% dense, gradebook 6.8%); our pathfinder room-0 measurement (2.26%).
- **Scored snapshots:** their `<score>.man` convention and catalog #34; our subdb.
- **Best-only economics:** their memorizer floor and free-submission play; our “submitting is free” doctrine.
- **The optimizer manifest:** their `optimize` tool is nearly clause for clause the “randomized highly performing Rust room optimizer with path-aware transformations” our team dictated mid-contest — theirs was running by 07:43 UTC day 3; ours became room\_ir/room\_place/ room\_compile after the end.
- **The Y delay-line:** our gpt built the architecture, our claude\_42 named the missing repair (counted delay, not spatial), their prototype implements exactly that repair.

The difference is never *what was understood*; it is *when the machinery existed*. They carried tools in; we derived tools live.

### 3. What they did that we genuinely did not

1. **Re-chose algorithms under process pressure.** Six problems,  $\geq 100\times$  each — pathfinder (3 generations), matmul (2), subset-sum (enumeration  $\rightarrow$  MITM), sudoku (bit-packing), plotter (measured cost model + planar re-plan), gradebook (roster ring). The scheduler’s rewrite mode institutionalizes this.
2. **Owned a compiler at contest time.** Generators + routers + finalizers per problem; `hand` actors with room unit tests; chokudai’s netlist CAD flow with nearest-port contract checking.
3. **Bought percent-scale gains with idle CPU.** The equivalence-gated annealer, auto-submitting, running unattended.
4. **Built knowledge for retrieval, not narrative.** Catalog + textbook + postmortem template; an agent (or human) mid-problem can *look up* “how do I demux without tags” (#19) or “how do I control vertical literals” (#30).
5. **Isolated the API key in a service** and versioned every submission with its event history.

### 4. What we did that held up

- **history-lesson: won outright** (6,561 vs 6,889) — our tokenizer + literal-packing line was world-class on the footprint-only problem.
- **tcp, memory, sort within 1.6–2.5 $\times$**  — protocol-level engineering competitive with theirs.
- **Operational throughput at the end:** 14 live improvements in the final hours through one judged submission relay, with ten of them being other agents’ finished work — the handoff topology worked.
- **Honest negative results discipline** mirrored theirs: our verified no-gos (llm line-merge deadlock, gradebook shape search) parallel their recorded rejections (memory 2-worker rotation).
- **Post-contest convergence:** our room compiler reached working form — rooms 4/5/3/1 of pathfinder compiled and verified in-machine — demonstrating the same architecture their tooling embodies.

### 5. Lessons folded forward (for 2027)

1. **Algorithm first, grid second — with a forcing function.** Adopt the 50/50 improve/rewrite split and open every solution document with the computation graph and its paper tick count.
2. **Arrive with the factory.** Extract this year’s judge/subdb/board scripts + the room compiler into a task-independent skeleton; rehearse once on a 2026 problem before the 2027 start.

3. **Conformance from hour one.** Local engine + official engine behind one CLI over one shared case file; divergence catalog as a living document.
4. **Generators as the deliverable.** Commit `<score>.py` next to every artifact; hand edits are for the frontier only, and get folded back into the generator the same day.
5. **Run the annealer as background income.** Equivalence-gated modifiers over valid artifacts, auto-submitting; ~1–6% per problem is a rank fraction at the margin.
6. **Leaderboard-driven scheduling** with expected-gain sampling (`2 - points` weighting) for grind-shaped work; keep our human-relay handoffs for design-shaped work.
7. **Knowledge as a catalog**, one technique per entry with named examples — grow it during the contest, not after.
8. **Floors are free points:** when a problem splits public/private cases, ship the honest memorizer while the real solution builds.

## 6. Verdict

Unagi's edge decomposes cleanly: **(a)** six algorithm rewrites worth  $100\times$ – $37,600\times$  each — human insight, scheduled and forced by process; **(b)** a compiler stack that made every machine regenerable, every constraint (nearest-pipe, literal pairing, capacity) checked at generation time; **(c)** infrastructure and knowledge systems amortized across years. Our team matched them on physics understanding, on operational tempo, and on one problem outright; we lost where we let the first working algorithm stand. The room compiler we built after the contest — and their entire repository, which reads as its independently-evolved twin — both point at the same 2027 doctrine: *generate, verify, anneal; and never stop asking whether the computation itself should change.*

---

## Appendix A — primary sources consulted

- `README.md`, `AGENTS.md`, `docs/preparation.md`, `docs/services.md`, `docs/architecture.md`, `docs/api/submissions.md`, `docs/optimizer.md`, `docs/optimizer-modifier-roadmap.md`, `docs/optimizer-inroom-plan.md`, `docs/simulator/README.md`, `docs/techniques/README.md` (56-entry catalog), `docs/textbook/`, `docs/glossary.md`, `docs/system-resource-guard.md`
- `solutions/<slug>/` postmortems and artifact trails for all 15 problem directories (~230 documents; ~40 read in full)
- `crates/` : simulator, optimizer, optimizer\_cli, hand, mandis, scorer, submit, best-answer, executor, www, editor-wasm
- `iwiwi/` : v1 work dirs, v2 curator/scheduler/solver, v3 loop + knowledge + prompts, v4-hand notes, editorial site, analysis
- `chokudai/` : README, compiler, dsl, gadgets; `wata/prompt.md`; `tos/` diaries
- Our archive: `archive/site-snapshot/standings/*.json`, `api/standings-overall.json` (2026-07-27 08:38 UTC)
- Working notes: `reports/unagi-analysis/notes/` in `tarstars/icfpc2026-claude`

*Report generated 2026-08-05 by the wheezards tooling agent (Claude). All quoted figures are from the cited files; translations from Japanese are ours.*